Cetus Limit Order

Audit Report

contact@movebit.xyz

https://twitter.com/movebit_

Tue Apr 16 2024

Cetus Limit Order Audit Report

1 Executive Summary

1.1 Project Information

Description	A limit order protocol
Type	Derivatives
Auditors	MoveBit
Timeline	Sun Apr 07 2024 - Tue Apr 16 2024
Languages	Move
Platform	Sui
Methods	Architecture Review, Unit Testing, Manual Review
Source Code	https://github.com/CetusProtocol/cetus-limitorder
Commits	0ad1830ae749b08a83163a83eec0ac0bd5e408ca 53bff8b64cc27047abd0869bed333406cc8108bf

1.2 Files in Scope

The following are the SHA1 hashes of the original reviewed files.

ID	File	SHA-1 Hash
ACL	sources/acl.move	6f56b0b734992a64241e7e5d539c3dc20b268671
SLU1	sources/skip_list_u128.move	5c21a0a8cc331a25c5e986b9bfec07b9d9047a9d
OU1	sources/option_u128.move	889289186ba07db849497ed809957f305f73e077
CON	sources/config.move	276000c02043c59d898d11c8f613ebf10c0fa842
ORD	sources/order.move	b4e3d09033b75db3777a33a040e873420cceb924

1.3 Issue Statistic

Item	Count	Fixed	Acknowledged
Total	5	5	0
Informational	0	0	0
Minor	2	2	0
Medium	0	0	0
Major	3	3	0
Critical	0	0	0

1.4 MoveBit Audit Breakdown

MoveBit aims to assess repositories for security-related issues, code quality, and compliance with specifications and best practices. Possible issues our team looked for included (but are not limited to):

- Transaction-ordering dependence
- Timestamp dependence
- Integer overflow/underflow by bit operations
- Number of rounding errors
- Denial of service / logical oversights
- Access control
- Centralization of power
- Business logic contradicting the specification
- Code clones, functionality duplication
- Gas usage
- Arbitrary token minting
- Unchecked CALL Return Values
- The flow of capability
- Witness Type

1.5 Methodology

The security team adopted the **"Testing and Automated Analysis"**, **"Code Review"** and **"Formal Verification"** strategy to perform a complete security test on the code in a way that is closest to the real attack. The main entrance and scope of security testing are stated in the conventions in the "Audit Objective", which can expand to contexts beyond the scope according to the actual testing needs. The main types of this security audit include:

(1) Testing and Automated Analysis

Items to check: state consistency / failure rollback / unit testing / value overflows / parameter verification / unhandled errors / boundary checking / coding specifications.

(2) Code Review

The code scope is illustrated in section 1.2.

(3) Formal Verification

Perform formal verification for key functions with the Move Prover.

(4) Audit Process

- Carry out relevant security tests on the testnet or the mainnet;
- If there are any questions during the audit process, communicate with the code owner in time. The code owners should actively cooperate (this might include providing the latest stable source code, relevant deployment scripts or methods, transaction signature scripts, exchange docking schemes, etc.);
- The necessary information during the audit process will be well documented for both the audit team and the code owner in a timely manner.

2 Summary

This report has been commissioned by Cetus to identify any potential issues and vulnerabilities in the source code of the Cetus Limit Order smart contract, as well as any contract dependencies that were not part of an officially recognized library. In this audit, we have utilized various techniques, including manual code review and static analysis, to identify potential vulnerabilities and security issues.

During the audit, we identified 5 issues of varying severity, listed below.

ID	Title	Severity	Status
CON-1	Version Check Error	Major	Fixed
CON-2	Missing Function to Modify `min_trade_amount`	Minor	Fixed
CON-3	Lack of Events Emit	Minor	Fixed
ORD-1	`repay_flash_loan` Permission Judgment Error	Major	Fixed
ORD-2	Arithmetic Overflow Error	Major	Fixed

3 Participant Process

Here are the relevant actors with their respective abilities within the Cetus Limit Order Smart Contract :

Administrator

- Administrators can update the value of `config.package_version` with the `update_package_version` function.
- Administrators can add tokens to the contract whitelist via the `add_into_token_list` function.
- Administrator can remove tokens from the contract whitelist using the `remove_from_token_list` function.
- Administrator can set the value of `config.deletion_grace_period` with the `set_config_deletion_grace_period` function.
- Administrator can set the value of `config.require_flash_loan_auth` with `set_config_require_flash_loan_auth` function.
- Administrator can set the value of `config.require_check_token_white_list` with the `set_config_require_check_token_white_list` function.
- Administrator can set roles for member with the `set_roles` function.
- Administrator can add a role for member with the `add_role` function.
- Administrator can remove a role for member with the `remove_role` function.
- Administrator can remove a member from contract with the `remove_role` function.

Keeper

- Keeper can cancel limit orders and delete canceled limit orders after limit order redemption has been completed or ended.
- Keeper can call `flash_loan` function and `repay_flash_loan` function for token exchange.

User

- Users can call the `place_limit_order` function to create a new limit order.
- Users can call the `create_rate_orders_indexer` function to create a new token indexer.

- Users can call the `create_indexer_and_place_limit_order` function to create a new token indexer and a new limit order.
- Users can call the `cancel_order_by_owner` function to cancel a limit order they have created.

4 Findings

CON-1 Version Check Error

Severity: Major

Status: Fixed

Code Location:

sources/config.move#136-139

Descriptions:

The implementation in the `checked_package_version` function, where it evaluates `VERSION >= config.package_version`, is incorrect. After upgrading the contract, if the `VERSION` increases while the `package_version` remains unchanged, this check can still pass.

```
pub fn checked_package_version(config: &GlobalConfig) {
    assert!(VERSION >= config.package_version, EPackageVersionDeprecate);
}
```

This implementation overlooks the necessity of ensuring that the `package_version` is updated alongside the contract upgrade, potentially allowing the check to pass erroneously.

Suggestion:

It is recommended to change the condition to `VERSION == config.package_version`.

Resolution:

This issue has been fixed. The client has changed the condition to `VERSION == config.package_version`.

CON-2 Missing Function to Modify `min_trade_amount`

Severity: Minor

Status: Fixed

Code Location:

sources/config.move#155-188

Descriptions:

All fields in the current `GlobalConfig` configuration are modifiable, except for `min_trade_amount`, which is not easily adjustable. To modify `min_trade_amount`, one must first call the `remove_from_token_list` function and then invoke `add_into_token_list`. This process is cumbersome. It would be advisable to introduce a direct functionality for modifying the `min_trade_amount` of tokens.

Suggestion:

It is recommended to introduce a direct functionality for modifying the `min_trade_amount` of tokens.

Resolution:

This issue has been fixed. The client has added the `update_min_trade_amount` function.

CON-3 Lack of Events Emit

Severity: Minor

Status: Fixed

Code Location:

sources/config.move#194-217

Descriptions:

The module lacks appropriate events for monitoring sensitive operations, which could make it difficult to track sensitive actions or detect potential issues. For example, `set_config_require_flash_loan_auth` , `set_config_require_check_token_white_list()` , `set_config_deletion_grace_period` , and so on.

Suggestion:

It is recommended to emit events for those important functions.

Resolution:

This issue has been fixed. The client has added events to these functions.

ORD-1 `repay_flash_loan` Permission Judgment Error

Severity: Major

Status: Fixed

Code Location:

sources/order.move#585-668

Descriptions:

In the `flash_loan` function, there's a conditional check:

```
if (get_config_require_flash_loan_auth(config)) {
    config::checked_keeper(config, sender);
};
```

This implies that the sender's `ACL_KEEPER` permission is checked only if `require_flash_loan_auth` is set to `true`. However, in the `repay_flash_loan` function, the sender's `ACL_KEEPER` permission is always checked. If `require_flash_loan_auth` is set to `false`, anyone can call the `flash_loan` function, but subsequently executing the `repay_flash_loan` function would fail due to this inconsistency. The validation at this point should align with that of the `flash_loan` function.

```
    public fun repay_flash_loan<PayCoin, TargetCoin>(
        config: &GlobalConfig,
        coin: &mut Coin<TargetCoin>,
        limit_order: &mut LimitOrder<PayCoin, TargetCoin>,
        receipt: FlashLoanReceipt<TargetCoin>,
        ctx: &mut TxContext
    ) {
        config::checked_package_version(config);
        let sender = tx_context::sender(ctx);
        config::checked_kepper(config, sender);
```

Suggestion:

It is recommended to adopt the same conditional check in the `repay_flash_loan` function as in the `flash_loan` function.

Resolution:

This issue has been fixed. The client adopted the same conditional check in the `repay_flash_loan` function as in the `flash_loan` function.

ORD-2 Arithmetic Overflow Error

Severity: Major

Status: Fixed

Code Location:

sources/order.move#586-635

Descriptions:

In the `flash_loan` function, if `remaining_pay_amount` equals 0 and `obtained_amount + target_repay_amount > total_expect_obtained_amount`, then the value of `target_repay_amount` will be `total_expect_obtained_amount - limit_order.obtained_amount`. This ensures that the sum of `obtained_amount` and `target_repay_amount` equals `total_expect_obtained_amount`. It means that `obtained_amount` and `target_repay_amount` can potentially exceed `total_expect_obtained_amount` (this maybe caused by rounding up). However, there's a problem being overlooked here. If the value of `remaining_pay_amount` has not yet decreased to 0, and the values of `obtained_amount` and `target_repay_amount` have already exceeded `total_expect_obtained_amount`, then after this call to the `flash_loan` function, the quantity of `obtained_amount` will exceed `total_expect_obtained_amount`. So when `remaining_pay_amount` reaches 0, it will consistently overflow, resulting in a failed function call.

```
let remaining_pay_amount = balance::value(&limit_order.pay_balance);
if (remaining_pay_amount == 0) {
    // the original order total expect obtained amount
    let total_expect_obtained_amount =
(full_math_u128::mul_div_ceil((limit_order.total_pay_amount as u128), PRECISION,
limit_order.rate) as u64);
    if (limit_order.obtained_amount + target_repay_amount >
total_expect_obtained_amount) {
        target_repay_amount = total_expect_obtained_amount -
limit_order.obtained_amount;
    };
    limit_order.obtained_amount = total_expect_obtained_amount;
} else {
```

```
    limit_order.obtained_amount = limit_order.obtained_amount + target_repay_amount;
};
```

Suggestion:

It is recommended to add a check for when `obtained_amount + target_repay_amount` exceeds `total_expect_obtained_amount`, not just when `remaining_pay_amount` equals 0.

Resolution:

This issue has been fixed. The client has already fixed it according to the recommended method.

Appendix 1

Issue Level

- **Informational** issues are often recommendations to improve the style of the code or to optimize code that does not affect the overall functionality.
- **Minor** issues are general suggestions relevant to best practices and readability. They don't post any direct risk. Developers are encouraged to fix them.
- **Medium** issues are non-exploitable problems and not security vulnerabilities. They should be fixed unless there is a specific reason not to.
- **Major** issues are security vulnerabilities. They put a portion of users' sensitive information at risk, and often are not directly exploitable. All major issues should be fixed.
- **Critical** issues are directly exploitable security vulnerabilities. They put users' sensitive information at risk. All critical issues should be fixed.

Issue Status

- **Fixed:** The issue has been resolved.
- **Partially Fixed:** The issue has been partially resolved.
- **Acknowledged:** The issue has been acknowledged by the code owner, and the code owner confirms it's as designed, and decides to keep it.

Appendix 2

Disclaimer

This report is based on the scope of materials and documents provided, with a limited review at the time provided. Results may not be complete and do not include all vulnerabilities. The review and this report are provided on an as-is, where-is, and as-available basis. You agree that your access and/or use, including but not limited to any associated services, products, protocols, platforms, content, and materials, will be at your own risk. A report does not imply an endorsement of any particular project or team, nor does it guarantee its security. These reports should not be relied upon in any way by any third party, including for the purpose of making any decision to buy or sell products, services, or any other assets. TO THE FULLEST EXTENT PERMITTED BY LAW, WE DISCLAIM ALL WARRANTIES, EXPRESS OR IMPLIED, IN CONNECTION WITH THIS REPORT, ITS CONTENT, RELATED SERVICES AND PRODUCTS, AND YOUR USE, INCLUDING BUT NOT LIMITED TO THE IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, NOT INFRINGEMENT.

